

>>>>

CLASSIFICADOR EMNIST BALANCED

Classificador Alfanumérico com Multi-Layer Perceptron (MLP) no Framework PyTorch

PUC-SP Vinicius de Lucena

DATASET

EMNIST BALANCED

- 47 Classes
- Flatten Para Escala 28x28

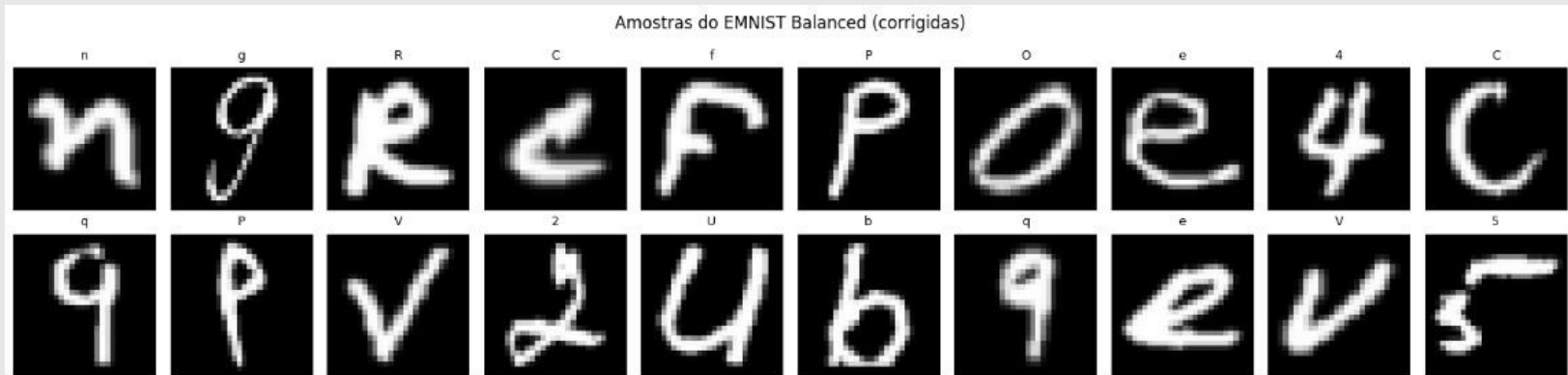
PRIMEIRO PASSO:

- Correção da Orientação das imagens originais do Dataset :



DATALOADER, TRANSFORM E SPLIT

- Cálculo de Média e Desvio para centralizar a escala dos pixels
- Realiza o Transform
- DataLoader com shuffle=True
- Split 90/10: 112.800 Treino (11.280 Validação)/18.800 Teste



01 ESTRATÉGIA:

MLP_A

Arquitetura básica (Baseline) com ReLU e Dropout 784 (28x28) -> 256 -> 128 -> 47

```
class MLP_A(nn.Module):
    """
    Arquitetura A: MLP simples com ReLU e Dropout.
    784 -> 256 -> 128 -> 47
    """

    def __init__(self, n_classes):
        super().__init__()
        self.rede = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, n_classes),
        )

    def forward(self, x):
        return self.rede(x)
```

MLP_B

Mesma arquitetura do MLP_A, porém com BatchNorm após cada camada, permitindo comparar o impacto da normalização por batch.

```
class MLP_B(nn.Module):
    """
    Arquitetura B: mesma topologia da A, mas com BatchNorm1d após
    cada camada linear. Permite comparar diretamente o impacto da
    normalizacao por batch.
    784 -> 256 -> 128 -> 47
    """

    def __init__(self, n_classes):
        super().__init__()
        self.rede = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, n_classes),
        )

    def forward(self, x):
        return self.rede(x)
```

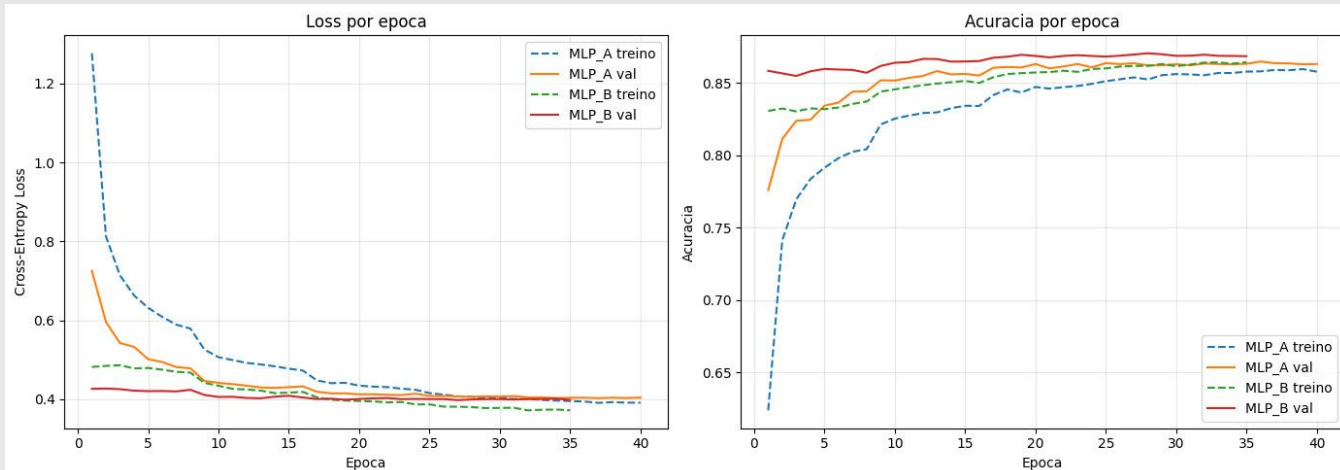
TREINAMENTO

Epoca 17		treino loss=0.4477	acc=0.8417		val loss=0.4189	acc=0.8606		[MELHOR]
Epoca 18		treino loss=0.4407	acc=0.8456		val loss=0.4149	acc=0.8612		[MELHOR]
Epoca 19		treino loss=0.4416	acc=0.8435		val loss=0.4150	acc=0.8608		[sem melhora 1/5]
Epoca 20		treino loss=0.4345	acc=0.8473		val loss=0.4123	acc=0.8632		[MELHOR]
Epoca 21		treino loss=0.4319	acc=0.8462		val loss=0.4124	acc=0.8602		[sem melhora 1/5]

Ambos os modelos foram treinados por **40 épocas** (com stop/patience de 5), arquitetura de loop, cada época passa por um DataLoader e devolve a loss e acurácia.

Hiperparâmetros:

- ADAM: Learning Rate $1e-3$ e Weight Decay de $1e-5$
- Scheduler: StepLR com size 8 por step

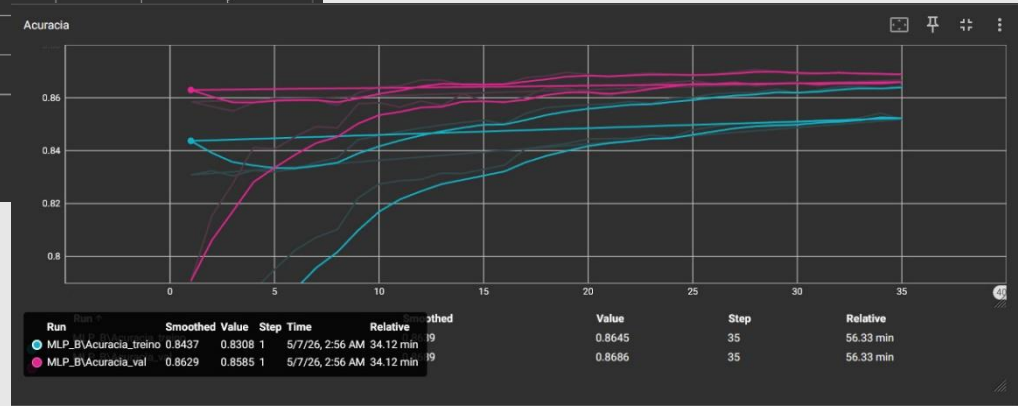


RESULTADOS DO TREINAMENTO (NOTEBOOK E TENSORBOARD)

MLP_A

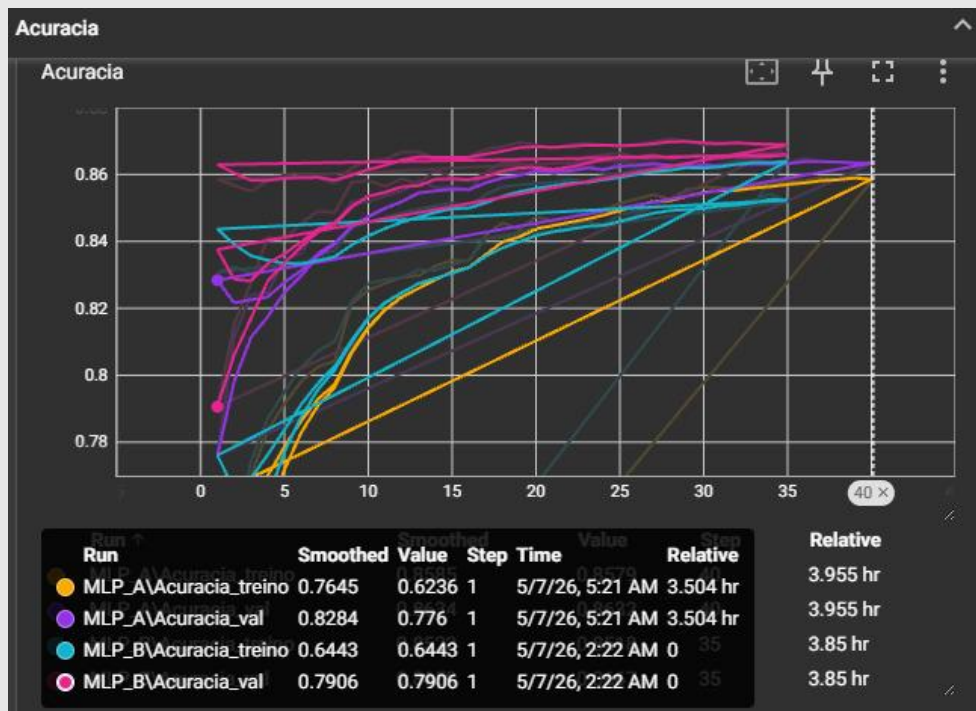


MLP_B



RESULTADOS DO TREINAMENTO (NOTEBOOK E TENSORBOARD)

MLP_A E MLP_B

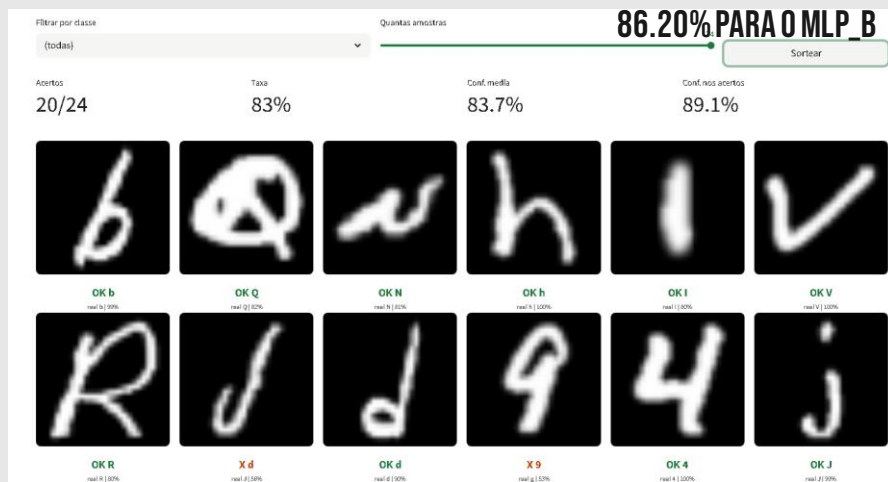


CHECKPOINTS E ACURÁCIA DOS MODELOS:

COM BASE NA ACURÁCIA E LOSS DAS ÉPOCAS, O MELHOR DESEMPENHO É GUARDADO LOCALMENTE, A FIM DE UTILIZAR O MELHOR VALOR. O MELHOR .PTH TAMBÉM É SALVO E VIRA A BASE DO MODELO NA APLICAÇÃO DO STREAMLIT.

CONJUNTO DE TESTE

COM O CONJUNTO DE TESTES, OBTIVEMOS UMA ACURÁCIA DE 85.87% PARA O MLP_A E

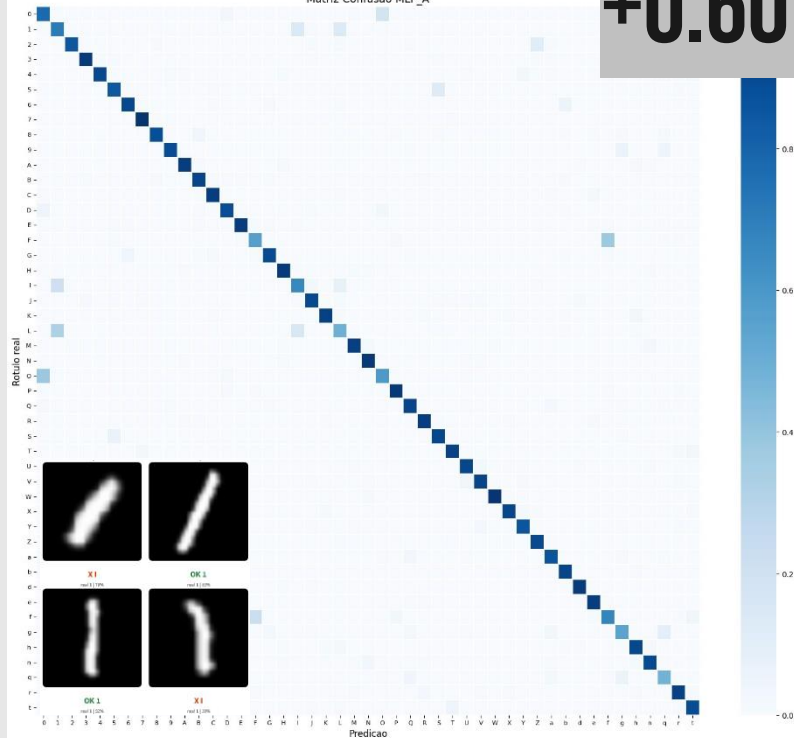


(IMAGEM DO APP)

MATRIZ DE CONFUSÃO

MLP_A

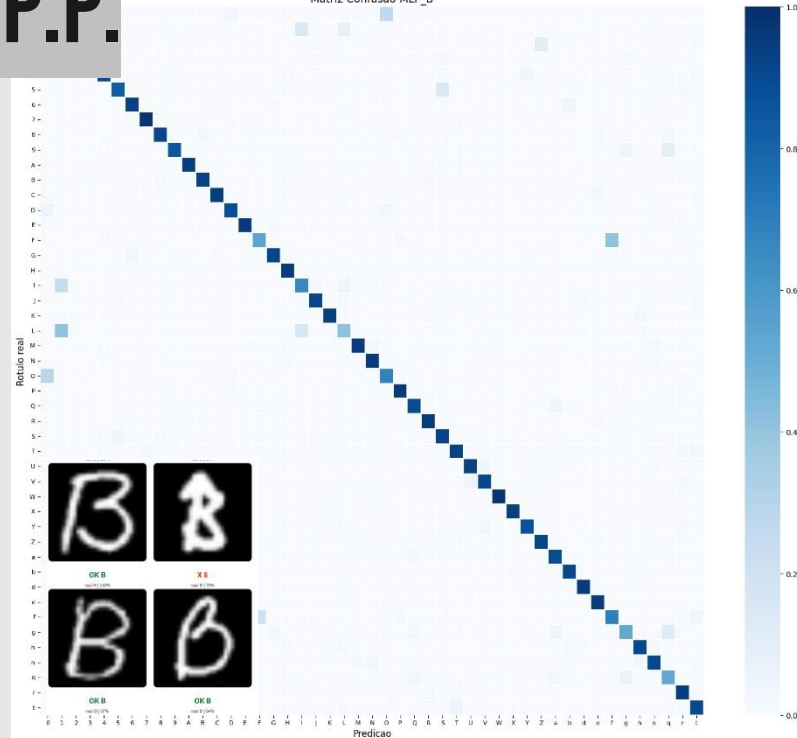
Matriz Confusao MLP_A



+0.60 P.P.

MLP_B

Matriz Confusao MLP_B



MAIORES ERROS EM CARACTERES AMBÍGUOS (B E 8, 1 E I, ETC.)

FEATURES (ENSEMBLE + TTA)

ENSEMBLE

Arquitetura básica (Baseline) com
ReLU e Dropout 784 (28x28) -> 256 ->
128 -> 47

+0.47 P.P.

✓ Test-time augmentation (TTA) ⓘ

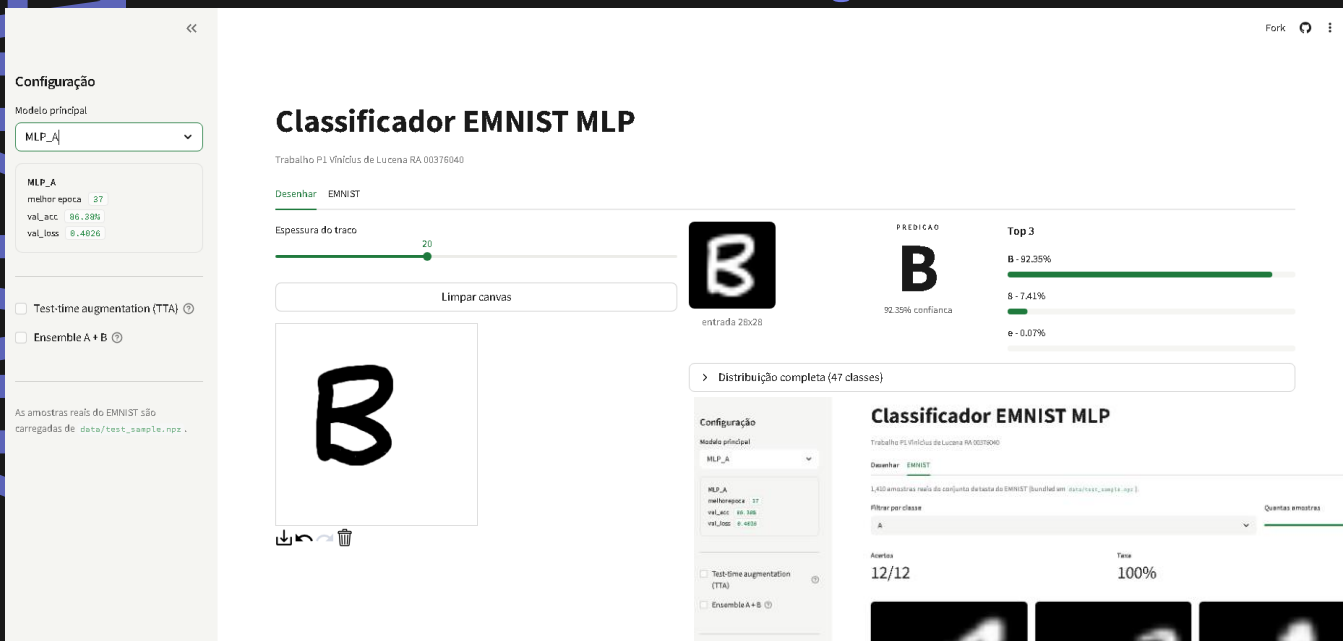
✓ Ensemble A + B ⓘ

TEST-TIME AUGMENTATION (TTA)

Mesma arquitetura do MLP_A, porém
com BatchNorm após cada camada,
permitindo comparar o impacto da
normalização por batch.

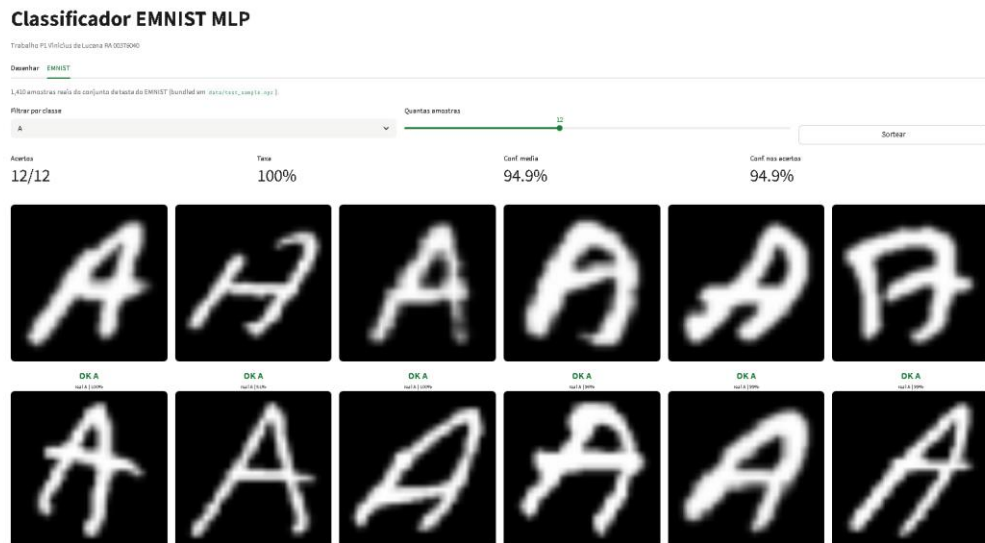
+0.16 P.P.

APLICAÇÃO STREAMLIT



OPÇÃO DE DESENHO, NA QUAL O
MODELO EXPÕE A CHANCE DE
CADA CLASSE

OPÇÃO DO DATASET: PEGA AMOSTRAS DO TESTE
E EXPÕE OS RESULTADOS.





OBRIGADO!

Vinicius de Lucena
RA: 00376040

